

Taller 8 - Personalización de QGIS con Python

Julián Andrés Torres Lozano

Tabla de contenidos

1	Taller 8 - Personalización de QGIS con Python	1
2	Ejercicio 1. Distancia elipsoidal con parada intermedia	2
2.1	Enunciado	2
2.2	Código ejecutado en QGIS	2
2.3	Resultado obtenido	2
2.4	Análisis del ejercicio	2
3	Ejercicio 2. Cambio de mensaje de información a advertencia	3
3.1	Enunciado	3
3.2	Código ejecutado en QGIS	3
3.3	Resultado obtenido	3
4	Ejercicio 3. Validación de la capa activa	4
4.1	Enunciado	4
4.2	Código ejecutado en QGIS	5
4.3	Resultado obtenido	5
5	Ejercicio 4. Cambio del CRS del proyecto	7
5.1	Código ejecutado en QGIS	7
5.2	Resultado obtenido	8

1. Taller 8 - Personalización de QGIS con Python

2. Ejercicio 1. Distancia elipsoidal con parada intermedia

2.1. Enunciado

Se debe calcular la distancia elipsoidal total entre San Francisco y Nueva York, considerando una parada intermedia en Las Vegas. Aplicar la clase `QgsDistanceArea` para calcular una distancia geodésica sobre el elipsoide WGS84 y convertir el resultado a kilómetros.

2.2. Código ejecutado en QGIS

```
from qgis.core import QgsDistanceArea, QgsPointXY, Qgs

san_francisco = (37.7749, -122.4194)
las_vegas = (36.1699, -115.1398)
new_york = (40.661, -73.944)

d = QgsDistanceArea()
d.setEllipsoid('WGS84')

sf = QgsPointXY(san_francisco[1], san_francisco[0])
lv = QgsPointXY(las_vegas[1], las_vegas[0])
ny = QgsPointXY(new_york[1], new_york[0])

distance_m = d.measureLine([sf, lv, ny])
distance_km = d.convertLengthMeasurement(distance_m, Qgs.DistanceUnit.Kilometers)

print(f"Distancia total: {distance_km:.2f} km")
```

2.3. Resultado obtenido

Al ejecutar el script en la consola de Python de QGIS, se obtuvo el siguiente valor:

```
Distancia total: 4271.03 km
```

2.4. Análisis del ejercicio

Este ejercicio permite evidenciar una de las funciones más importantes del núcleo espacial de QGIS: la medición geodésica sobre un elipsoide. A diferencia de una distancia euclidiana

plana, la clase `QgsDistanceArea` permite incorporar la curvatura terrestre cuando se define un modelo geodésico como WGS84.

El aspecto más importante del código es la transformación de las coordenadas a objetos `QgsPointXY`. Aunque las coordenadas suelen expresarse como (`latitud`, `longitud`), QGIS requiere el orden (`X`, `Y`), es decir (`longitud`, `latitud`). Este detalle es crítico, ya que si se invierte mal el orden, el cálculo resultará incorrecto.

El método `measureLine()` recibe una lista de puntos y calcula la distancia acumulada entre ellos. En este caso, el resultado representa el recorrido San Francisco → Las Vegas → Nueva York. Posteriormente, la distancia se convierte de metros a kilómetros mediante `convertLengthMeasurement()`.

3. Ejercicio 2. Cambio de mensaje de información a advertencia

3.1. Enunciado

Se debía modificar el mensaje mostrado en la barra de mensajes de QGIS para que apareciera como una advertencia (`Warning`) y, adicionalmente, que desapareciera automáticamente después de 5 segundos.

3.2. Código ejecutado en QGIS

```
from qgis.core import Qgs

iface.messageBar().pushMessage(
    'Advertencia',
    'Este mensaje se cerrará en 5 segundos',
    level=Qgis.Warning,
    duration=5
)
```

3.3. Resultado obtenido

Al ejecutar el script en la consola de Python de QGIS, se mostró correctamente un mensaje de advertencia en la barra superior de la interfaz. El mensaje permaneció visible durante 5 segundos y luego desapareció automáticamente.

La correcta aparición de la barra amarilla confirma que el método `pushMessage()` fue utilizado adecuadamente. En este caso, el ejercicio demuestra que PyQGIS puede emplearse no solo para análisis espacial, sino también para mejorar la comunicación con el usuario.

La Figura 1 muestra la ejecución correcta del ejercicio en QGIS.

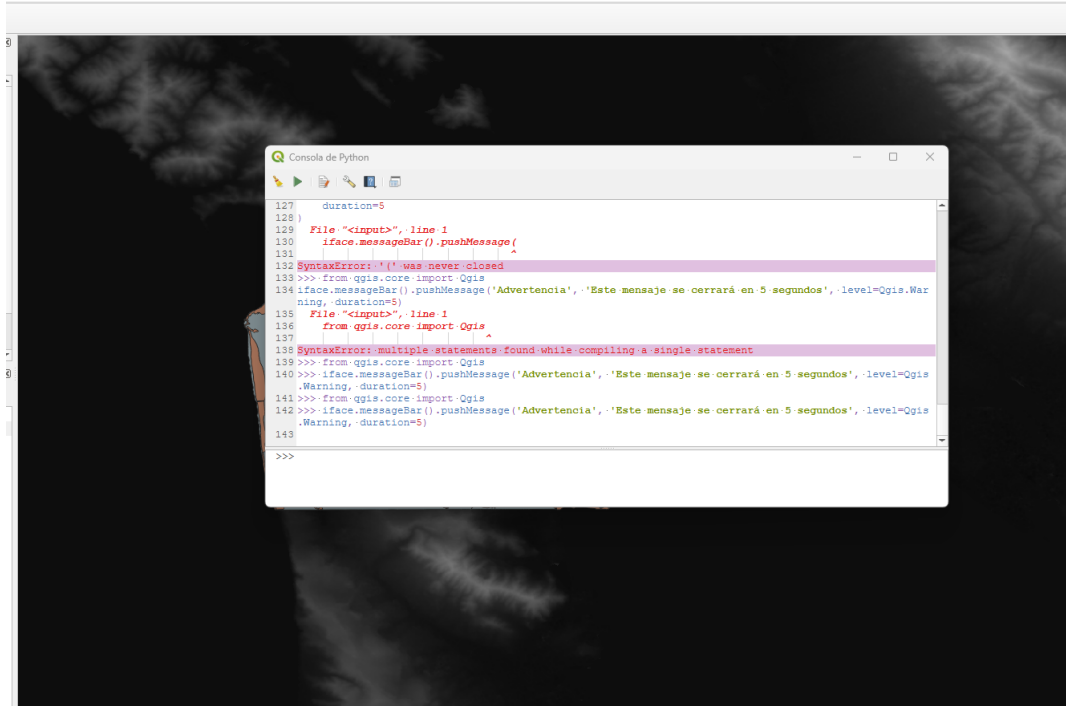


Figura 1: Mensaje de advertencia mostrado en la barra superior de QGIS

4. Ejercicio 3. Validación de la capa activa

4.1. Enunciado

Se debía escribir un fragmento de código que verificara si la capa activa seleccionada por el usuario se denominaba `sf_zoning`. Si la condición se cumplía, debía mostrarse un mensaje de éxito; en caso contrario, un mensaje de error.

4.2. Código ejecutado en QGIS

```
from qgis.core import Qgis

layer = iface.activeLayer()

if layer is None:
    iface.messageBar().pushMessage(
        'Error',
        'No hay una capa activa seleccionada.',
        level=Qgis.Critical,
        duration=5
    )
elif layer.name() == 'sf_zoning':
    iface.messageBar().pushMessage(
        'Éxito',
        'La capa activa es sf_zoning.',
        level=Qgis.Success,
        duration=5
    )
else:
    iface.messageBar().pushMessage(
        'Error',
        f'La capa activa es "{layer.name()}" y no "sf_zoning".',
        level=Qgis.Critical,
        duration=5
    )
```

4.3. Resultado obtenido

Al ejecutar el script en QGIS, el sistema identificó que la capa activa seleccionada era `seismic_zones` y no `sf_zoning`, por lo que mostró correctamente un mensaje de error en la barra superior de la interfaz.

La Figura [Figura 2](#) muestra la validación realizada en QGIS.

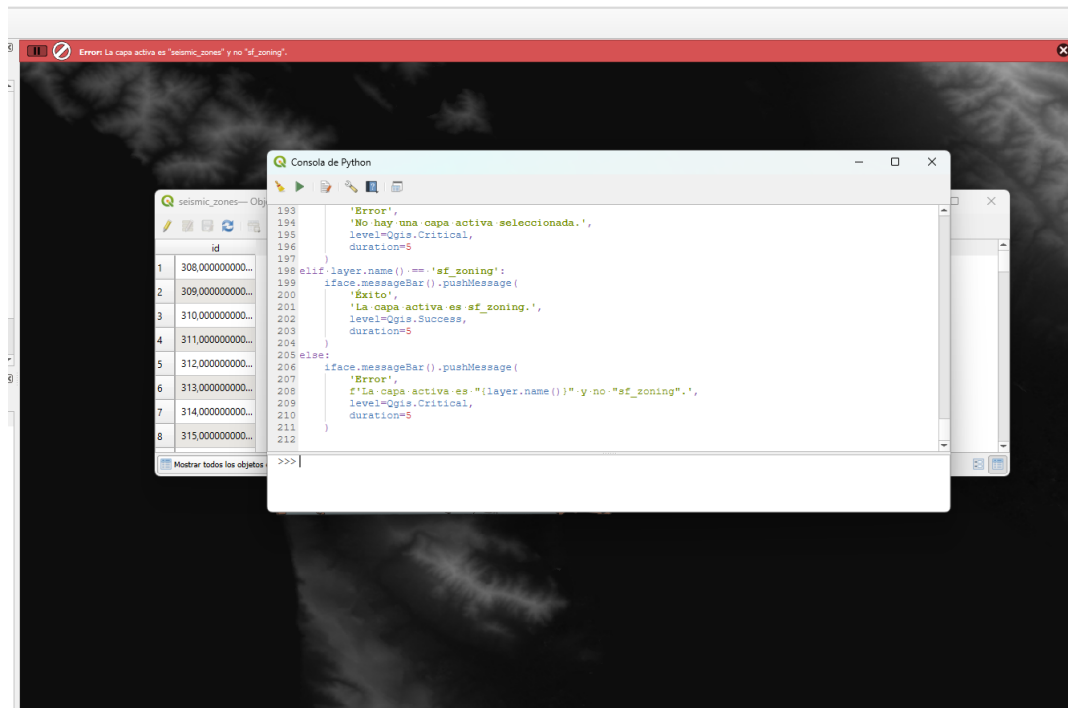


Figura 2: Validación de la capa activa en QGIS mostrando que la capa seleccionada es seismic_zones y no sf_zoning

5. Ejercicio 4. Cambio del CRS del proyecto

Se construyó una interfaz simple en QGIS que permita modificar programáticamente el CRS del proyecto actual.

5.1. Código ejecutado en QGIS

```
from PyQt5.QtWidgets import QLabel, QLineEdit, QPushButton
from qgis.core import QgsProject, QgsCoordinateReferenceSystem, Qgs

crsToolbar = iface.addToolBar('CRS Toolbar')

label = QLabel('Ingrese un código EPSG (ej: 3116):', parent=crsToolbar)
crsTextBox = QLineEdit('4326', parent=crsToolbar)
crsTextBox.setFixedWidth(80)
button = QPushButton('Establecer!', parent=crsToolbar)

crsToolbar.addWidget(label)
crsToolbar.addWidget(crsTextBox)
crsToolbar.addWidget(button)

def changeCrs():
    input_text = crsTextBox.text().strip()

    if not input_text.isdigit():
        iface.messageBar().pushMessage(
            'Error',
            f'El valor "{input_text}" no es un código EPSG válido.',
            level=Qgis.Critical,
            duration=5
        )
        return

    crs = QgsCoordinateReferenceSystem(f'EPSG:{input_text}')

    if not crs.isValid():
        iface.messageBar().pushMessage(
            'Error',
            f'No se pudo crear el CRS EPSG:{input_text}.',
            level=Qgis.Critical,
            duration=5
        )
```

```

    )
    return

QgsProject.instance().setCrs(crs)
iface.mapCanvas().refresh()

iface.messageBar().pushMessage(
    'Éxito',
    f'CRS del proyecto cambiado a EPSG:{input_text}.',
    level=Qgis.Success,
    duration=5
)

button.clicked.connect(changeCrs)

```

5.2. Resultado obtenido

Al ejecutar el script en QGIS, se creó correctamente una barra de herramientas denominada **CRS Toolbar**, compuesta por una etiqueta descriptiva, una caja de texto para ingresar el código EPSG y un botón para aplicar el cambio del sistema de referencia espacial del proyecto.

La Figura 3 muestra la creación de la barra de herramientas para el cambio del CRS en QGIS.

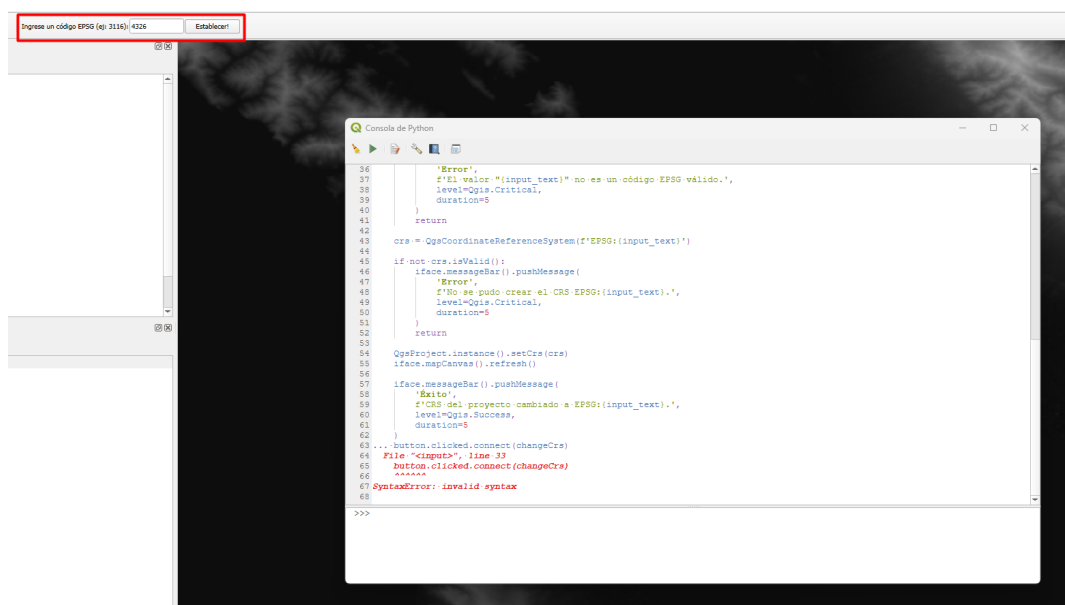


Figura 3: Barra de herramientas creada para ingresar un código EPSG y cambiar el CRS del proyecto